



Universidade Federal do Rio Grande do Norte

Alice in the Uberland

Ian Gabriel, Lucas Apolônio, Eduardo Marinho

2026-09-11

1 Contest

2 Grafos

3 Strings

Contest (1)

template.cpp13 lines

```
#include<bits/stdc++.h>
using namespace std;

// dependendo da lib mudar para ll
#define int long long
#define pii pair<int, int>
#define all(x) x.begin(), x.end()
#define vi vector<int>
#define rep(i, a, b) for(int i=a; i<(b); i++)

signed main() {
    ios_base::sync_with_stdio(false);cin.tie(NULL);
}
```

hash.sh3 lines

```
# Hashes a file, ignoring all whitespace and comments. Use for
# verifying that code was correctly typed.
cpp -dD -P -fpreprocessed | tr -d '[:space:]' | md5sum |cut -c-6
```

Grafos (2)

2.1 Matching

MaximumMatching.cpp
Description: Matching máximo de um grafo bipartido. n e m são os tamanhos das partes. mt[i] é vértice da parte A que deu match com i (i é da parte B).
Time: O(VE)

```
vector<int> g[MAXA];
bool used[MAXA];
int mt[MAXB], n, m;

bool findPath(int x) {
    if(used[x]) return false;
    used[x] = true;
    for(int y : g[x]) {
        if(mt[y] == -1 || findPath(mt[y])) {
            mt[y] = x;
            return true;
        }
    }
    return false;
}

int maximumMatching() {
    for(int i=0; i<m; i++) mt[i] = -1;
    int ans = 0;
    for(int i=0; i<n; i++) {
        for(int i=0; i<n; i++) used[i] = false;
        ans += findPath(i);
    }
    return ans;
}
```

Strings (3)

3.1 Automatos

AhoCorasick.cpp
Description: Autômato de Aho-Corasick. lnk[i] é o suffix link de i. go[i][j] é o automato. ex[i] é o proximo vertice terminal seguindo pelos links de i, e nmatches[i] é a quantidade de matches do estado i.
Usage: start no começo, adiciona as strings, depois rode o build
Time: Inserção O(|S|). Build O(Σ|Si|k)

```
namespace Aho {
    int lnk[MAX], go[MAX][K], ex[MAX], nmatches[MAX];
    bool output[MAX];
    int tam;

    void start() {
        tam = 1;
        memset(go, -1, sizeof(go));
        memset(lnk, -1, sizeof(lnk));
        memset(ex, -1, sizeof(ex));
    }

    void add_string(string &t) {
        int r = 0;
        for(char ch : t) {
            int c = valor(ch);
            if(go[r][c] == -1) {
                go[r][c] = tam; tam++;
            }
            r = go[r][c];
        }
        output[r] = nmatches[r] = true;
    }

    void build() {
        queue<int> q;
        lnk[0] = ex[0] = 0;
        for(int i=0; i<K; i++) {
            if(go[0][i] == -1) go[0][i] = 0;
            else {
                lnk[go[0][i]] = ex[go[0][i]] = 0;
                q.push(go[0][i]);
            }
        }
        for(; !q.empty(); q.pop()) {
            int v = q.front();
            int u = lnk[v];
            for(int i=0; i<K; i++) {
                if(go[v][i] == -1) go[v][i] = go[u][i];
                else {
                    lnk[go[v][i]] = go[u][i];
                    int w = lnk[go[v][i]];
                    ex[go[v][i]] = (output[w] ? w : ex[w]);
                    nmatches[go[v][i]] += nmatches[w];
                    q.push(go[v][i]);
                }
            }
        }
    }
}
```

KMP.cpp
Description: Função PI e matching de padrão. pi[i] é o maior j (diferente de i), tal que o prefixo de tamanho j é igual ao sufixo de tamanho j da string s[0..i]. matching(pat, t) usa pi para calcular as posições em que pat ocorre em t
Time: pi = O(|s|); matching = O(|pat| + |t|)

```
vi pi(string s) {
    vi pi(s.size()); pi[0] = 0;
    for(int i=1, j=0; i<s.size(); i++) {
        while(j && s[i] != s[j]) j = pi[j-1];
        if(s[i] == s[j]) j++;
        pi[i] = j;
    }
    return pi;
}

vi matching(string &pat, string &t) {
    vi p = pi(pat), match;
    for(int i=0, j=0; i<t.size(); i++) {
        while(j && t[i] != pat[j]) j = p[j-1];
        if(t[i] == pat[j]) j++;
        if(j == pat.size()) match.push_back(i-j+1), j = p[j-1];
    }
    return match;
}
```